Monitoring Roadmap

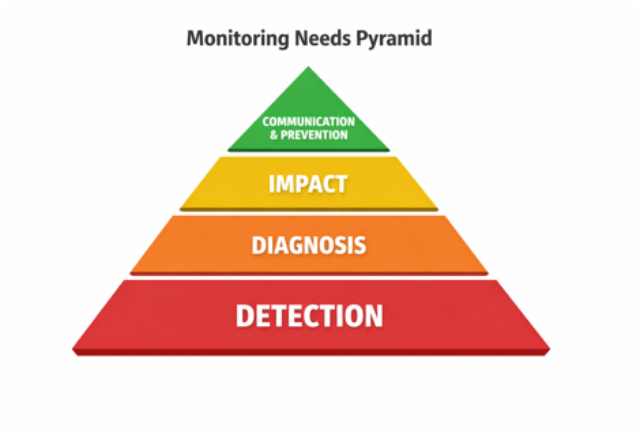
1. Objectives

The objective of this document is to draft a simple and actionable roadmap to set up more observability regarding our key assets as a Data Engineering team (data, platform, pipelines, ...)

This is a document written to better prepared our action plan for Q4 2025 PI.

2. The monitoring needs pyramid

When an incident happens we usually go through 4 phases:



1. **Detection**: did something broke? What assets broke?
2. **Diagnosis**: why and where it broke?

3. **Impact**: what assets (data, pipelines, ...) are impacted and what's the criticity level?
4. **Communication and prevention**: what's the expected downtime, who are the impacted users, what information should be shared?

Today the main pain point of the team is the **detection** phase. We are not equipped enough to quickly detect that something broke. We usually get to know it too lately, when the impacts are bigger.

3. What are the assets to be monitored

Put simply, in our context, 5 key assets should be monitored:

1. **Platform**

Type of questions platform monitoring helps answer:

- What is the cost of pipeline runs/storage at any given time?
- What are the clusters consuming the most ressources ? Who run them ?
- Are we over/under provisioning clusters ?

2. **Users**

Type of questions users monitoring helps answer:

- Who ran which job/query/notebook?
- Which tables are the most queried?
- Are user permissions aligned with actual usage?

3. **Data Connectors**

As a general rule, data connectors are maintained by the data engineering team (elasticsearch is the only exception). Those are the applications in charge of extracting the data from the source systems to the raw landing zone. They usually live outside of Databricks (e.g. AWS Firehose, Fivetran, ...).

Type of questions data connector monitoring helps answer:

- Is my connector down ?
- Is the data there, yes or no ?

- Does data volume arriving in raw make sense ?

4. **Databricks Pipelines**

Type of questions pipeline monitoring helps answer:

- Did the pipeline fail ? Does it silently fail?
- When it fails, do we know why?
- Are pipelines becoming slower over time?
- Are we silently skipping/missing data?

5. **Data**

Type of questions data monitoring helps answer:

- Did columns distributions shift over time ?
- Is today's data comparable to historical data ?
- Did producers violate implicit data contracts ?
- Which downstream assets are now compromised?

Pipeline monitoring will be our focus here because this is where the data get transformed.

A preliminary work has been done by the team and had the objective to list all existing monitoring tools available in our tech stack, their strengths and limitations: 🔍 Existing monitoring capabilities

4. Pipeline Health

Pipeline health = The degree to which a pipeline ran when expected, completed successfully, and produced usable data on time.

The idea is to define for each of our pipelines a set of **validation rules** that provides, when evaluated, a visual status about the pipeline behaviour:

- 🟢 : Everything is green

- 🟠 : Pipeline/Connector not broken but attention is required
- 🔴 : Error detected Pipeline/Connector broken

The goal is essentially to list simple sanity checks that hits a balance to provide confidence in a pipeline's health

5. Pipeline Inventory

Pipeline	Description	Tasks	Schedule
ElasticSearch V1	Transforms Mongo master data (cameras, users, subscriptions)	1. runs cross reference V1 (user identification, skey creation and quarantine) 2. transforms data from raw to bronze (Hive Metastore) (Hive Metastore) 3. transforms data from bronze to silver (Hive Metastore)	• Daily • 4 AM EST • Runs in ~2-3h
Cross Reference V2	Cross-reference process uniquely identifying a spypoint and vosker user by matching identifiers (email, mongo_id, and stripe_id) into a skey. Unlike V1 which is a subtask of the whole ElasticSearch V1 pipeline, Cross Reference V2 is an independent pipeline, which is triggered by ElasticSearch V2	1. matches unique identifiers (email, mongo_id and stripe_id) 2. creates skey through encryption_key 3. filters out unidentified users into quarantine	• N/A (Triggered by ElasticSearch V2)
ElasticSearch V2	Unlike V1 this version uses the cross reference V2, which is an enhanced version that's less restrictive and leads to less unidentified records ending in quarantine	1. triggers cross reference V2 (skey creation and quarantine) 2. transforms data from raw to bronze (Unity Catalog)	• Daily • 4:15 AM EST • Runs in ~1h

Exchange rate	Creates daily exchange rates for Stripe revenue pipeline	1. Extracts CAD, EUR, GBP exchange rates from Netsuite 2. Save data to gold (Hive Metastore)	• Daily • 7 AM EST • Runs in ~10min
Stripe	Transforms Stripe data from raw straight to silver	1. transforms stripe data (spypoint/spypointcom and vosker/voskercom) from raw to bronze (Hive Metastore) 2. transforms stripe data (spypoint/spypointcom and vosker/voskercom) from bronze to silver (Hive Metastore)	• Daily • 6:45 AM EST • Runs in ~1-2h
Revenue	Calculate RPA from Stripe data	1. calculates RPA 2. saves to gold (Hive Metastore)	• Daily • 8:15 AM EST • Runs in ~30min-1h
EDA	Transforms raw EDA data (subscriptions, photos, ...) from raw to bronze Temporary: also in charge of silver reconciliation table for specific ecommerce tactical solution	1. decodes encoded events 2. drops PII data 3. saves decoded events to bronze (Unity Catalog) 4. temporary: creates reconciliation table and save to silver (Unity Catalog)	• Daily • 6:00 AM EST • Runs in ~30min-1h
Data Vault	Foundational silver layer	1. creates a staging layer from all bronze data sources 2. creates reconciled data vault layer (called Core) with objects ready to be consumed (users, subscriptions, cameras, ...)	• N/A (schedule definition still in progress)
[AD HOC] Unity Catalog Silver Generation	Ad hoc pipeline that copies all data sources previously saved from Hive Metastore's bronze mount	1. loads data from bronze by data source 2. saves data by source to silver (Unity	• Daily • 8:15 AM EST • Runs in ~2h

	to Unity Catalog's silver layer	Catalog)	
[AD HOC] KPI Torpedo on bronze	Ad hoc pipeline that creates a bronze V1 view called "all bronze" based on 3 index snapshots (10, 20 and last day of the month). This view is further used to derive all the Torpedo KPIs used by the BI team. This view was the result of large ES index volume that led to performance issues bypassed by this sampling method	1. Creation of all bronze view 2. Calculation of all Torpedo KPIs	• Monthly - 1st day of the month • 11:00 AM EST • Runs in ~1h

6. Validation Rules

A validation rule is a set of conditions about data integrity or pipeline execution stats that gives an overall pipeline health status. The goal is to define for each pipeline a set of validation rules. Usually each pipeline already provides a bunch of useful data about it's own execution statuses and ingestion logging that are easily retrievable to produce those rules.

Notes:

- Raw is a specific layer that acts as a landing zone. For most of our use cases, this layer is populated by connectors living outside of Databricks, and are maintained by the Data Engineering team (e.g. EDA: Firehose, Stripe: Fivetran, ElasticSearch: Logstash). To monitor the behaviour of these connectors 3 approaches are possible:

    1. Include them in the central monitoring dashboard as if they were Databricks pipelines. We would use "proxy validation rules" to give health statuses. For example:

        -Is yesterday's data present in raw layer? → If yes, it's green. If no, it's red. The connector might be down, go to connector's in-platform monitoring dashboard to further investigate the issue

-Is yesterday's data arriving late compared to L7D avg? → If no, it's green. If yes, it's red. The connector is up but it might need a cluster upsize.

2. Export connectors logs to S3 or directly to Databricks. For example, for AWS-based connectors, Cloudwatch subscription filters let you export logs to any destination for further analysis.

3. Use API calls from Databricks to connector's platform to fetch health status data

4. Fully check connector's health status through in-platform monitoring tools

- **For connectors, our choice it's to go for option 1. Even though it doesn't give a full picture of the connector health, it satisfies the basic need for incident detection and it helps centralize our monitoring in one place.**

Asset	Validation Rules
	Pipelines
ElasticSearch V1	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • Did the pipeline actually processed the most recent index ? • Did the number of unique rows/identifiers in raw indexes decreased (<=5% L7D moving avg) ?
Cross Reference V2	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD
ElasticSearch V2	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD

Exchange rate	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD
Stripe	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD
Revenue	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD
EDA	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD
Data Vault	• Did the pipeline run? • Did the pipeline run with success? • Did the pipeline run duration on range (<=5% L7D moving avg) ? • TBD
[AD HOC] Unity Catalog Silver Generation	• TBD
[AD HOC] KPI Torpedo on bronze	• TBD
Connectors	
Stripe	• Did we receive yesterday data per table (e.g. account, invoice, ...) ?
Elastic	• Did we receive yesterday data per index type (user, cam, subs)? • Is yesterday's index size anormal per index type (user, cam, subs)? (<=5% L7D moving avg)
EDA	• Did we receive yesterday data per topic? • Does the total size of avro files received yesterday anormal per topic? (<=5% L7D moving avg)

	• Do we have records in error under the records/ folder per topic ?

7. Central Monitoring Dashboard Mockup

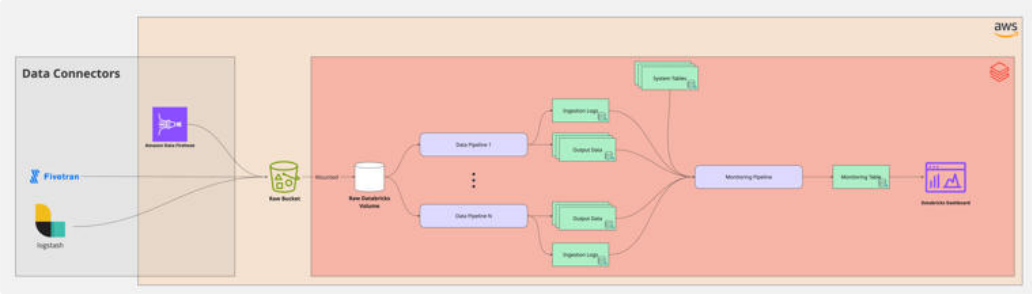
Evaluate these validation rules on a daily basis to give a pipeline's health status history. In the first phase, manuel daily check up is enough, alerting is not necessary. Giving a visual status comes with several advantages:

- defines a common ground within the data engineering team about what's green and red
- centralizes all our pipeline monitoring checks in one place vs having them scattered in different places (slack, teams channels, databricks notebooks, ...)
- shareable across other data teams to be more proactive can greatly improve our ability to better detect, diagnose and communicate issues.

Asset	Health Status	Validation Rules
Pipelines		
ElasticSearch V1	🟢	Validation Rule 1 🟢 Validation Rule 2 🟢
Cross Reference V2	🔴	Validation Rule 1 🔴 Validation Rule 2 🟢
Exchange Rate	🟠	Validation Rule 1 🟠 Validation Rule 2 🟢
ElasticSearch V2	...	...
Stripe	...	...
Revenue	...	...
EDA	...	...
Data Vault	...	...
[AD HOC] Unity Catalog Silver Generation	...	...
[AD HOC] KPI Torpedo on bronze	...	...
Connectors		

Stripe	🟢	Validation Rule 1 🟢 Validation Rule 2 🟢
EDA	🔴	Validation Rule 1 🔴 Validation Rule 2 🟢
Elastic	...	...

8. High-level Solution Architecture

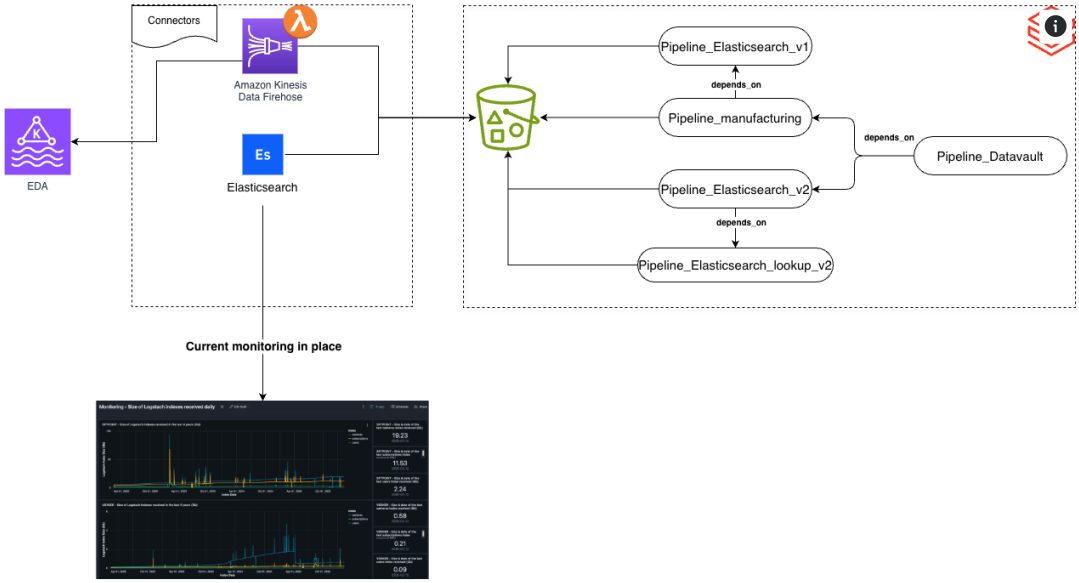


High-Level Solution Architecture Diagram

Source: https://miro.com/app/board/uXjVGVvedDE=/?share_link_id=493429862915 Connectez votre compte Miro

9. Pipelines and connectors priority

Today we have a strong dependency between several pipelines using the same data sources. This has allowed us to establish certain priorities for monitoring pipelines first.



As a starting point, we have a Databricks dashboard that monitors Elastic indexes. We can use this dashboard to monitor the connectors and pipelines that consume this data.

Below is a list of connectors and pipeline to tackle:

1. Firehose connector
2. Lambda function
3. ElasticSearch V1
4. ElasticSearch V2
5. Cross Ref V2
6. Manufacturing
7. Data Vault

Among these pipelines, one in particular presents a higher risk of failure than the others since it handles streams coming from Kafka. In addition to data from Kafka, it also needs data from ElasticSearch V1.

From this, we can consider the manufacturing pipeline as our **first priority**. This implies monitoring the:

- Firehose Connector
- Lambda function used inside firehose

- ElasticSearch V1 (pipeline status and schema evolution)
- Manufacturing (bronze layer)

As a **second priority**, we can tackle Data Vault, which consumes data from EDA and ElasticSearch V2.

The table below describes the priority and dependencies between existing pipelines and connectors.

Pipeline/Connector	Dependencies	Priority
Firehose connector	N/A	**P0**
Lambda function	• Firehose connector	**P0**
ElasticSearch V1	• Elastic connector	**P1**
Manufacturing	• Firehose connector • Pipeline ElasticSearch V1	**P2**
Cross Ref V2	• Elastic connector	**P3**
ElasticSearch V2	• Elastic connector	**P3**
Data vault	• Pipeline Manufacturing • Pipeline ElasticSearch V2 • Stripe	**P4**
Stripe	N/A	**P5**

10. Next Steps

1. ~~Validate the scope of pipelines and connectors to monitor~~
2. For each pipeline and connector define the set of basic validation rules in scope
3. For each rule/pipeline what should be the action if an alert is triggered
4. Create the Bitbucket repo
5. Code implementation:
    a. Validation rule definitions → Output: table and config that define validation rules per pipeline/connector
    b. Validation rule evaluation → Output: table and config that evaluate the validation rules per pipeline/connector and calculate the expected thresholds
    c. Daily health status evaluation → Output: Aggregate rule results per pipeline/connector per day
6. Tests
    a. IT and/or UT

7. Deployment:
    a. CI on bitbucket
    b. Bundles and schedules
    c. deployment tests
8. Manual dashboard creation on Databricks